

Compare the Triplets:-

Code:-

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;
class Result {
    public static List<Integer> compareTriplets(List<Integer> a, List<Integer> b) {
int alice = 0;
        int bob = 0;
        for (int i = 0; i < 3; i++) {
            if (a.get(i) > b.get(i)) {
                alice++;
            } else if (a.get(i) < b.get(i)) {
                bob++;
            }
        }
        List<Integer> result = new ArrayList<>();
        result.add(alice);
        result.add(bob);
        return result;
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));
        List<Integer> a = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
            .map(Integer::parseInt)
            .collect(toList());
        List<Integer> b = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
            .map(Integer::parseInt)
            .collect(toList());
        List<Integer> result = Result.compareTriplets(a, b);
        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining(" "))
                + "\n"
        );
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

}

Output:-

✓ **Test case 0**

✓ Test case 1 

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

✓ Test case 6 

Compiler Message

Success

Input (stdin)

1	5 6 7
2	3 6 10

[Download](#)

Expected Output

1	1 1
---	-----

[Download](#)

Contains Duplicate:-

Code:-

```
import java.util.*;

class Solution {
    public boolean containsDuplicate(int[] nums) {
        HashSet<Integer> set = new HashSet<>();
        for (int num : nums) {
            if (set.contains(num)) {
                return true;
            }
            set.add(num);
        }
        return false;
    }
}
```

Output:-

The screenshot displays the LeetCode submission interface for the 'Contains Duplicate' problem. The submission status is 'Accepted' with 79/79 testcases passed. The code is written in Java and is shown in the editor. The test result shows the input [1,2,3,1] resulting in 'true'.

Runtime: 18 ms | Beats 76.89%

Memory: 108.21 MB | Beats 37.97%

Testcase: Accepted Runtime: 0 ms

Case 1: Input: nums = [1,2,3,1] Output: true Expected: true